

Étape 5

Automatisation et supervision

Faire tourner l'application en continu, livrer le code sans intervention, surveiller la production

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	2 octobre 2026

Sommaire

1. Objectif de l'étape

2. Nos choix en bref

3. Notions de base

3.1 Trois raisons pour qu'une application change

3.2 Orchestration et DAG

3.3 Intégration, livraison et déploiement continus

3.4 Supervision

4. Architecture

5. Automatisation avec Airflow

5.1 Les trois traitements

5.2 Airflow orchestre, il ne calcule pas

5.3 Choix de calendrier

5.4 Ingestion continue

5.5 Contrôle qualité

5.6 Le bot tourne en continu

6. Réentraînement : champion contre challenger

6.1 Principe

6.2 Étapes

6.3 Registre des modèles

6.4 Réentraînement anticipé

6.5 Résultats

7. Intégration et déploiement continus

7.1 GitHub Actions

7.2 Déploiement par Watchtower

7.3 Un défaut rencontré

8. Supervision

8.1 Les composants

8.2 Pourquoi un exportateur

8.3 Règles d'alerte

8.4 Le tableau de bord

9. Limites

10. Annexe : fichiers

1. Objectif de l'étape

L'étape 5 demande que l'application fonctionne en continu, sans que personne ne lance de commande. Elle comprend trois volets : automatiser les étapes précédentes, mettre en place une chaîne d'intégration continue pour mettre à jour l'application, et superviser la production.

Demande	Réalisation	Emplacement
Automatiser les étapes précédentes	3 traitements Airflow : collecte et bot, dérive, réentraînement	<code>airflow/dags/</code>
Chaîne d'intégration continue	GitHub Actions : 5 contrôles, publication des images, déploiement par Watchtower	<code>.github/workflows/ci.yml</code>
Superviser la production	Prometheus, 12 règles d'alerte, sondes, tableau de bord Grafana	<code>monitoring/</code> , <code>api/exportateur.py</code>

2. Nos choix en bref

Choix	Pourquoi	Alternative écartée
Airflow orchestre, il ne calcule pas	chaque tâche lance un script du projet, testé ; une panne se reproduit avec une ligne de commande	mettre la logique dans les DAG
Exécuteur local, collecte décalée d'une minute	peu de tâches ; le décalage évite d'arriver avant la clôture de la bougie	Celery et sa file de messages
Champion contre challenger	un modèle réentraîné n'est pas forcément meilleur ; il ne remplace l'ancien que s'il fait au moins aussi bien sur 21 jours non vus	remplacer le modèle à chaque réentraînement
Registre MLflow, alias <code>champion</code>	garde la trace de ce qui a été essayé et de ce qui sert	des fichiers sans historique
CI GitHub Actions en 5 contrôles	analyse, tests, DAG, configuration, déploiement complet ; un DAG cassé disparaît d'Airflow sans message	publier sans vérifier
Watchtower en mode pull	la machine va chercher les images ; rien n'est ouvert vers l'extérieur	un exécuteur GitHub sur la machine : dépôt public, du code d'un fork pourrait y tourner
Prometheus et Grafana	Prometheus interroge chaque service : un service arrêté se voit tout de suite	consulter les journaux à la main
Un exportateur à part	l'état du pipeline (données, bot, réentraînement) est en base ; il le traduit en métriques et survit à une panne de l'API	ne superviser que l'API

3. Notions de base

3.1 Trois raisons pour qu'une application change

Une application de machine learning évolue pour trois raisons : son **code** change, de nouvelles **données** arrivent, et son **modèle** vieillit à mesure que le marché s'éloigne de celui qu'il a appris. Chacune demande sa propre automatisation :

Ce qui change	Déclencheur	Outil
Le code	un envoi sur GitHub	intégration et déploiement continu (GitHub Actions, Watchtower)
Les données	le temps qui passe	orchestration (Airflow)
Le modèle	le temps qui passe, ou une dérive	orchestration (Airflow) et registre de modèles (MLflow)

La supervision (Prometheus et Grafana) observe les trois.

3.2 Orchestration et DAG

Un **orchestrateur** lance des traitements selon un calendrier et dans un ordre donné, relance ceux qui échouent et garde l'historique de chaque exécution. Airflow décrit un traitement sous forme de **DAG** (graphe orienté acyclique) : un ensemble de **tâches** reliées par des dépendances, sans boucle. Le **planificateur** décide quand lancer chaque tâche ; l'**exécuteur** la fait tourner.

3.3 Intégration, livraison et déploiement continu

- L'**intégration continue** (CI) vérifie automatiquement chaque modification du code : analyse, tests, construction. Une modification qui casse quelque chose est signalée avant d'être utilisée.
- La **livraison continue** produit à chaque modification validée un artefact prêt à être installé ; ici, des images Docker publiées sur un registre.
- Le **déploiement continu** (CD) met cet artefact en service sans intervention humaine.

3.4 Supervision

Superviser, c'est mesurer en permanence l'état de l'application et être prévenu quand il se dégrade. Une **métrique** est une mesure numérique horodatée (nombre d'appels, durée, retard des données). Une **règle d'alerte** est une condition sur une métrique, par exemple « plus aucune bougie depuis trois intervalles ». Une **sonde** appelle un service de l'extérieur, comme le ferait un utilisateur, pour vérifier qu'il répond vraiment.

5. Automatisation avec Airflow

5.1 Les trois traitements

DAG	Calendrier	Tâches
<code>cryptobot_collecte</code>	toutes les 15 minutes, à 1, 16, 31 et 46	ingestion des bougies, contrôle qualité, calcul des variables, tour du bot
<code>cryptobot_derive</code>	chaque jour à 6 h 30 UTC	mesure de la dérive, réentraînement anticipé si elle est forte
<code>cryptobot_reentrainement</code>	chaque dimanche à 3 h UTC	extrait figé, nouveau modèle, comparaison, publication

5.2 Airflow orchestre, il ne calcule pas

Chaque tâche lance une commande du projet (`python -m scripts.ingestion_continue`, `python -m scripts.reentrainer entrainer...`), exactement celle que l'on lancerait à la main. Le DAG ne contient que le calendrier, l'ordre et la politique de nouvelles tentatives. La logique reste dans les scripts, où elle est testée, et une panne se reproduit en recopiant une ligne de commande.

Ces commandes tournent dans un environnement Python séparé de celui d'Airflow, installé dans la même image. Airflow fige les versions de centaines de bibliothèques ; scikit-learn et MLflow en demandent d'autres. Deux environnements évitent tout conflit, et le modèle est entraîné avec la version exacte de scikit-learn que l'API utilise pour le charger.

Airflow tourne avec l'exécuteur local (*LocalExecutor*) : les tâches sont des processus lancés par le planificateur. Un exécuteur distribué comme Celery ajouterait une file de messages et des machines de calcul, inutiles pour quelques tâches toutes les 15 minutes.

5.3 Choix de calendrier

- **Décalage d'une minute.** Une bougie de 15 minutes se clôture à la 14e minute et 59 secondes. Lancer à l'heure pile risque d'arriver avant sa clôture.
- **Pas de rattrapage des exécutions manquées.** L'ingestion repart toujours de la dernière bougie en base ; un seul passage comble n'importe quel retard. Au premier lancement, elle a rattrapé 25 jours (15 700 bougies) en 15 secondes.
- **Une exécution à la fois par DAG.** Deux collectes simultanées écriraient les mêmes bougies.
- **Nouvelles tentatives espacées.** Trois essais, avec un délai qui double à chaque fois : la plupart des échecs sont passagers (Binance indisponible un instant, base qui redémarre). Le contrôle qualité fait exception : relancer un contrôle ne change pas son constat.

5.4 Ingestion continue

L'ingestion applique les principes de l'étape 2 : la réponse brute est archivée dans MongoDB avant toute transformation, chaque bougie de PostgreSQL garde sa référence vers ce document, et l'écriture est idempotente. Seules les bougies clôturées sont écrites. Une tâche relancée par Airflow après un échec ne crée aucun doublon.

5.5 Contrôle qualité

Un modèle nourri de bougies manquantes ou incohérentes produit des prédictions fausses sans lever d'erreur. À chaque passage, trois vérifications par paire et par pas de temps :

Vérification	Alerte	Échec
Fraîcheur de la dernière bougie	plus de 3 intervalles de retard	plus de 12 intervalles
Complétude sur 2 jours	au moins une bougie manquante	
Cohérence		un plus haut sous le plus bas, un volume négatif

Chaque résultat est enregistré dans la table `controles_qualite`, et un échec fait échouer la tâche, donc apparaît dans Airflow.

5.6 Le bot tourne en continu

À l'étape 4, le carnet de positions n'avancait que lorsque quelqu'un ouvrait l'interface. Désormais, Airflow appelle l'API toutes les 15 minutes pour chacune des 30 combinaisons (5 paires, 3 pas de temps, 2 styles), avec sa propre clé. Un tour complet prend environ 90 secondes. Si l'application a été arrêtée, l'API rejoue au premier appel les bougies manquées depuis la dernière évaluée, avec les mêmes règles.

6. Réentraînement : champion contre challenger

6.1 Principe

Un modèle réentraîné n'est pas forcément meilleur : période d'apprentissage atypique, données manquantes, erreur introduite dans le code. Remplacer systématiquement le modèle en service par le plus récent dégraderait la production sans que personne ne le remarque. Nous mettons donc les deux modèles en concurrence : le modèle en service (le **champion**) et le nouveau (le **challenger**) sont mesurés sur la même période, que le challenger n'a jamais vue. Le challenger ne remplace le champion que s'il fait au moins aussi bien.

6.2 Étapes

Étape	Règle	Raison
Figer	extrait des deux dernières années, signé par une empreinte SHA-256, vérifiée avant l'entraînement	on n'entraîne jamais sur des données vivantes ; le résultat doit être reproductible
Entraîner	recette du modèle final de l'étape 3 : forêt aléatoire, calibration, deux seuils par style	le duel compare des données récentes, pas deux méthodes
Évaluer	les 21 derniers jours, retirés de l'apprentissage, avec un jour d'écart	l'étiquette d'une bougie dépend de la suivante : sans cet écart, la fin de l'apprentissage verrait le début de l'évaluation
Décider	au moins 30 ordres, et une accuracy au moins égale à celle du champion (style conservateur)	en dessous de 30 ordres, la mesure n'est pas fiable
Publier	copie du nouveau fichier à côté de l'ancien, puis renommage ; l'ancien est archivé	l'API ne lit jamais un fichier à moitié écrit, et un retour en arrière reste possible

Le renommage d'un fichier est une opération **atomique** : l'API voit l'ancien fichier ou le nouveau, jamais un mélange. Elle détecte le changement de date du fichier et recharge le modèle à l'appel suivant. La référence utilisée pour la dérive est régénérée en même temps, pour décrire les données du modèle en service.

6.3 Registre des modèles

MLflow enregistre chaque réentraînement : paramètres, mesures du champion et du challenger, décision et raison. Chaque challenger devient une **version** du modèle `cryptobot-direction` dans le registre. L'**alias** `champion` désigne la version en service et ne se déplace qu'à la publication. Le registre montre ainsi à la fois ce qui a été essayé et ce qui a servi.

6.4 Réentraînement anticipé

Le DAG de dérive déclenche un réentraînement sans attendre dimanche quand au moins 8 séries sur 15 sont en dérive forte. Un garde-fou l'empêche de le faire plus d'une fois tous les trois jours : une dérive forte ne disparaît pas en une nuit, et sans cette limite elle provoquerait un réentraînement chaque matin sur des données presque identiques.

6.5 Résultats

Date	Champion	Challenger	Décision
23 septembre	0,575 (447 ordres)	0,558 (539 ordres)	champion gardé
26 septembre (déclenché par la dérive)	0,594	0,571	champion gardé

Les écarts sont dans les marges d'erreur (par exemple 0,529 à 0,620 contre 0,516 à 0,600 le 23 septembre) : les deux modèles se valent, et à valeur égale ou incertaine la règle garde celui qui est déjà en service. Relancé trois fois le même jour, le duel a donné les mêmes chiffres à la quatrième décimale, ce que garantit l'extrait figé.

Ces duels ont eu lieu alors que les 15 séries surveillées étaient en dérive forte. Le modèle en service gardait pourtant 57,5 % de bonnes réponses, dans la marge des 60 % mesurés à l'étape 3. Une dérive des données ne s'accompagne pas forcément d'une baisse de performance : c'est pourquoi la dérive déclenche une comparaison, et non une mise en service automatique.

7. Intégration et déploiement continu

7.1 GitHub Actions

À chaque envoi sur la branche principale, cinq contrôles tournent en parallèle sur des machines de GitHub. Les images ne sont construites et publiées que si tous réussissent. L'ensemble dure environ 5 minutes.

Contrôle	Ce qu'il vérifie
Analyse statique	erreurs visibles sans exécuter le code : nom inconnu, import inutilisé, variable jamais lue (ruff)
Tests unitaires	les 149 tests, avec la mesure de couverture
Intégrité des DAG	chaque DAG se charge dans Airflow 3.3, avec les tâches dans l'ordre prévu
Configuration	règles d'alerte Prometheus (<code>promtool</code>) et configuration nginx (<code>nginx -t</code>) valides
Déploiement complet	toute l'application démarrée de zéro sur une machine vierge, avec des secrets neufs, puis vérifiée de l'extérieur

Un DAG qui ne se charge pas disparaît silencieusement de l'interface d'Airflow : la collecte s'arrêterait sans message d'erreur visible. Le contrôle d'intégrité des DAG existe pour cette raison. Il a d'ailleurs refusé une version dès le premier jour, lorsque Airflow 3.3 a changé la façon de charger les DAG.

Les images `api`, `interface` et `airflow` sont publiées sur le registre de GitHub (`ghcr.io`), étiquetées du commit qui les a produites.

7.2 Déploiement par Watchtower

Watchtower est un service qui surveille le registre toutes les 5 minutes. Quand une image utilisée par un conteneur a une nouvelle version, il télécharge l'image, arrête le conteneur, le recrée à l'identique avec la nouvelle image et supprime l'ancienne. Seuls les conteneurs marqués par une étiquette sont concernés (API, interface, exportateur, Airflow) ; les bases de données ne le sont jamais.

La chaîne complète devient : envoi sur GitHub, contrôles verts, image publiée, mise en service automatique.

Le déploiement fonctionne en mode **pull** : c'est la machine qui va chercher les mises à jour. Rien n'est ouvert vers l'extérieur, et GitHub n'exécute jamais de code sur cette machine. L'alternative, un exécuteur GitHub installé sur la machine, a été écartée : le dépôt est public, et du code proposé depuis un fork pourrait y être exécuté.

Les images du registre sont privées (règle de l'organisation). Watchtower s'authentifie avec un jeton GitHub limité à la lecture des images (`read:packages`), stocké dans `.env` .

7.3 Un défaut rencontré

Au premier déploiement, Watchtower a remplacé les six conteneurs construits localement par les images de la CI. Une image construite par Docker Compose porte les étiquettes qui permettent à Compose de retrouver ses conteneurs ; en passant à l'image de la CI, qui ne les a pas, Watchtower les a retirées, et Compose ne reconnaissait plus ces conteneurs. Ils ont été recréés par Compose. Sur une machine de production, on ne construit donc pas d'image localement : on part de celles de la CI.

8. Supervision

8.1 Les composants

Composant	Rôle
Prometheus	lit les métriques toutes les 15 secondes, évalue les règles d'alerte, garde 30 jours d'historique
API	expose ses métriques : appels et durée par route, erreurs, décisions prises, refus de clé
Exportateur	lit l'état du pipeline en base et le traduit en métriques
Sondes (blackbox)	appellent chaque page web comme un utilisateur : interface, API à travers le proxy, Airflow, MLflow
Grafana	tableau de bord de production, décrit en fichier et chargé au démarrage

Prometheus fonctionne en mode **pull** : il interroge chaque service, les services n'envoient rien. Un service arrêté se voit immédiatement, puisqu'il ne répond plus.

8.2 Pourquoi un exportateur

Les questions les plus importantes en production ne concernent pas l'API : les données arrivent-elles, le bot tourne-t-il, le dernier réentraînement a-t-il eu lieu ? Les réponses sont en base, écrites par Airflow. Un petit service, construit sur l'image de l'API, les lit et les traduit en métriques, ce qui permet à Prometheus de déclencher des alertes dessus. Il tourne à part pour survivre à une panne de l'API, et garde ses lectures en cache une minute pour ne pas charger la base.

8.3 Règles d'alerte

Question	Alertes	Niveau
Le service répond-il ?	API injoignable, page web en panne, plus de 5 % d'erreurs, réponses lentes	critique
Les données arrivent-elles ?	plus de bougie depuis 3 intervalles, contrôle qualité en échec, base illisible	critique
Le bot tourne-t-il ?	aucun tour de carnet depuis 45 minutes	critique
Le modèle est-il encore adapté ?	dérive forte généralisée, dérive non mesurée depuis 2 jours, aucun réentraînement depuis 9 jours	avertissement
Quelqu'un force-t-il l'accès ?	plus de 30 appels par minute refusés pour clé invalide	avertissement

Chaque alerte doit durer avant de se déclencher (de 2 minutes à 1 heure selon le cas) : un redémarrage de quelques secondes ne doit pas en produire. Les règles sont écrites dans un fichier versionné (`monitoring/prometheus/alertes.yml`) et vérifiées par la CI.

8.4 Le tableau de bord

Le tableau « CryptoBot, production » regroupe : l'état de chaque service, les alertes en cours, le trafic et les temps de réponse de l'API, le retard des données par série, l'évolution de la dérive, les décisions du bot, le taux de trades gagnants et le gain net moyen, le capital virtuel et l'historique des réentraînements. Il est décrit dans un fichier JSON versionné : il est identique sur toute machine qui démarre l'application.

Grafana lit l'historique directement dans PostgreSQL avec un compte en **lecture seule** : un tableau de bord n'a aucune raison de pouvoir modifier une donnée.

9. Limites

Limite	Conséquence	Piste
Alertes sans notification	elles se consultent dans Grafana et Prometheus, personne n'est prévenu	ajouter Alertmanager, vers un courriel ou une messagerie
Une seule machine	si elle s'arrête, tout s'arrête ; le carnet se rattrape au redémarrage	héberger sur un serveur, avec sauvegarde des volumes
Pas de HTTPS	acceptable tant que tout reste sur <code>127.0.0.1</code>	terminaison TLS devant nginx avant toute ouverture au réseau
Secrets dans un fichier <code>.env</code>	protégés par les droits du fichier seulement	un coffre à secrets en production
Modèle non rentable	l'automatisation entretient un modèle qui ne couvre pas ses frais	changer de cible, pas seulement de données

10. Annexe : fichiers

Fichier	Rôle
<code>airflow/Dockerfile</code> , <code>airflow/dags/</code>	image Airflow et les trois DAG
<code>scripts/ingestion_continue.py</code>	bougies manquantes, vers MongoDB et PostgreSQL
<code>scripts/controle_qualite.py</code>	contrôle qualité des données récentes
<code>scripts/reentrainer.py</code>	extrait figé, champion contre challenger, publication
<code>.github/workflows/ci.yml</code>	intégration et publication des images
<code>monitoring/</code>	configuration de Prometheus, des sondes et de Grafana
<code>api/exportateur.py</code> , <code>api/metriques.py</code>	métriques du pipeline et de l'API
<code>docker-compose.yml</code>	les 16 services, leurs profils et leurs réseaux